Intitulé du cours:
Technique des applications web

Chapitre 2 :
Développer avec React JS

Filière: 2éme année – bts web

Enseignante: Abid Arij

Année de formation 2025-2026

React JS

Plan

- ❑ Présentation générale du ReactJS
- ❑ Les fondamentaux du React
- ❑ Caractéristiques principales
- ❑ Environnement de développement
- ❑ Structure d’une application React
- ❑ Architecture des applications Web
- ❑ Le DOM et le Virtual DOM
- ❑ Data Binding en React

Qu’est-ce que React ?

React est une **bibliothèque** JavaScript utilisée pour créer des interfaces utilisateur interactives.

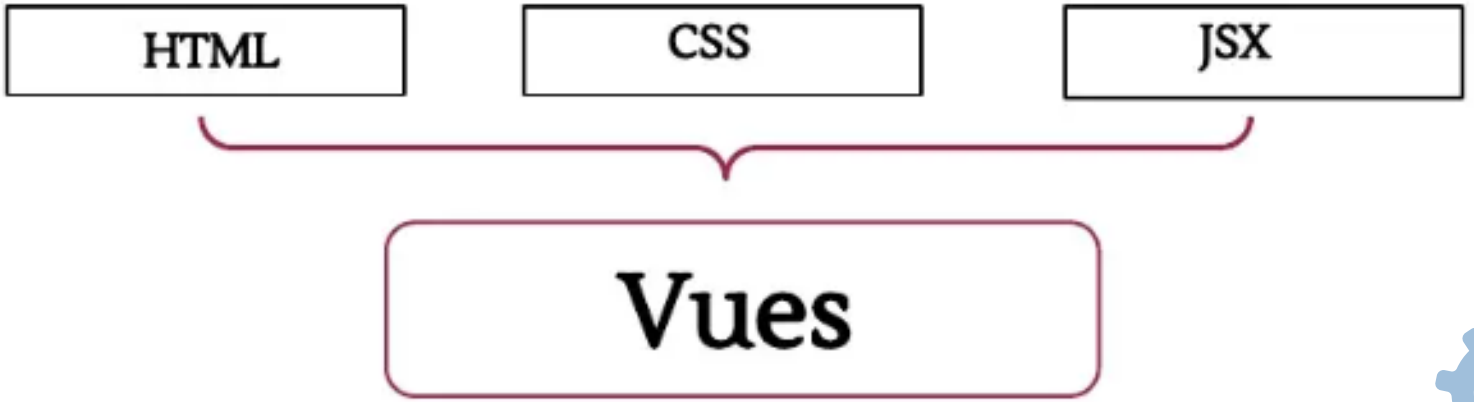
- Elle a été développée par **Facebook** en 2013 et est utilisée dans des applications connues comme Instagram et WhatsApp Web.

Principe fondamental :

- Découper l’interface en composants réutilisables.
- Optimiser l’affichage et les mises à jour grâce au Virtual DOM.

Quels langages utilise React ?

- HTML pour les vues.
- CSS pour les styles.
- JSX (JavaScript XML): pour les scripts



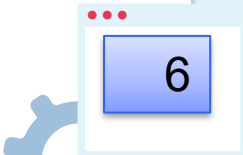
Caractéristiques principales

- **Basée sur les composants** : chaque partie de l'interface (bouton, menu, formulaire…) est un composant réutilisable.
- **JSX** : un langage qui mélange HTML et JavaScript dans le même fichier pour plus de clarté.
- **Écosystème riche:** React est **léger et flexible** mais fonctionne avec un **écosystème complet** : routing, state management, UI libraries, etc.

- **Virtual DOM** : React met à jour uniquement les éléments nécessaires dans la page, ce qui rend l'application très rapide.

Caractéristiques principales

- **Facile à tester**
  - Les composants étant indépendants, ils sont faciles à **tester et déboguer**.
- **Single Page Application (SPA):** React permet de créer des SPA où **seules les parties qui changent sont mises à jour**, sans recharger toute la page.
- **Unidirectional Data Flow** (flux de données unidirectionnel) : les données circulent d'un parent vers ses enfants.



Sites web développés avec React :

facebook

TERADEK

Instagram

NETFLIX

KHANACADEMY

reddit

asana

airbnb

Définition d'une Library (Librairie)

Une **librairie** est un **ensemble de fonctions ou composants réutilisables** que tu appelles quand tu en as besoin.

- **Tu contrôles le flux du programme.**
- La librairie intervient uniquement quand TU l'appelles.

Avec React :

- Tu décides l'architecture
- Tu choisis les outils (routing, gestion d'état, etc.)
- Tu appelles React pour afficher les composants

Définition d'un Framework

Un **framework** est une **structure complète de développement** qui impose une manière d'organiser le code.

Un **framework** est une structure complète qui fournit :

- architecture
- règles
- conventions
- outils intégrés

👉 **Le framework contrôle le programme.**

Express JS

NEXT.JS

Vue.js

Angular

Angular est un **framework complet**.
Il inclut :
•Routing intégré
•Dependency Injection
•Gestion d'état
•Formulaires
•HTTP Client
•Architecture MVC/MVVM
•CLI officiel

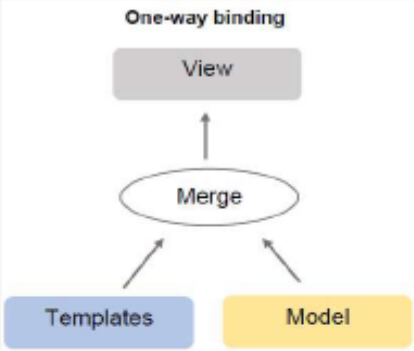
Différence fondamentale

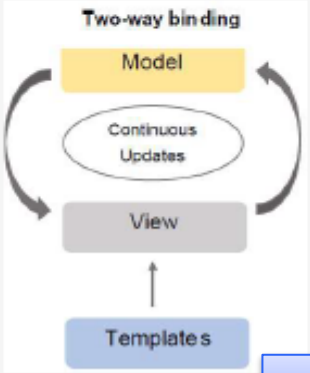
Library	Framework
Tu appelles la librairie	Le framework appelle ton code
Tu contrôles le flux	Le framework contrôle le flux
Flexible	Structuré
Plus léger	Plus complet
Exemple : React	Exemple : Angular

Angular Vs React :

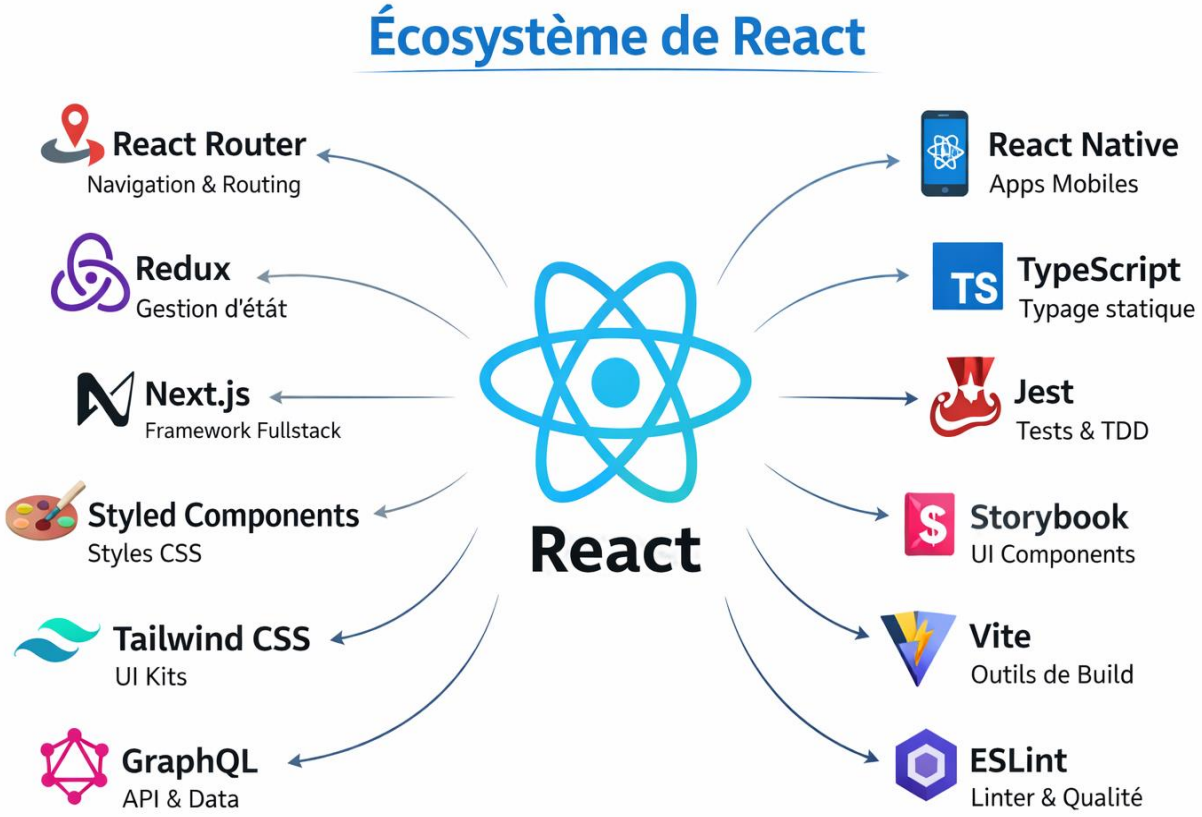
- **Angular Vs React :**

	Angular	React
Type	Framework	Librairie
Auteur	google	Facebook
Apparition	Octobre 2010	Mai 2013
Langage	TypeScript	JavaScript (JSX)
Data binding	Bi-directionnel	Uni-directionnel
Du web au mobile	Ionic	React Native
Usage	Sony, nike, Genral motors, HBO, …	Netflix, whatsapp, dropbox, instgram, …

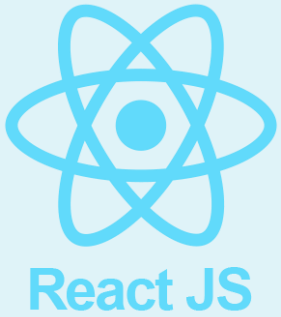




L’**écosystème de React** représente **l’ensemble des outils, bibliothèques et technologies** qui travaillent avec **React** pour développer des applications web (et mobiles).



Environnement du travail et Installation

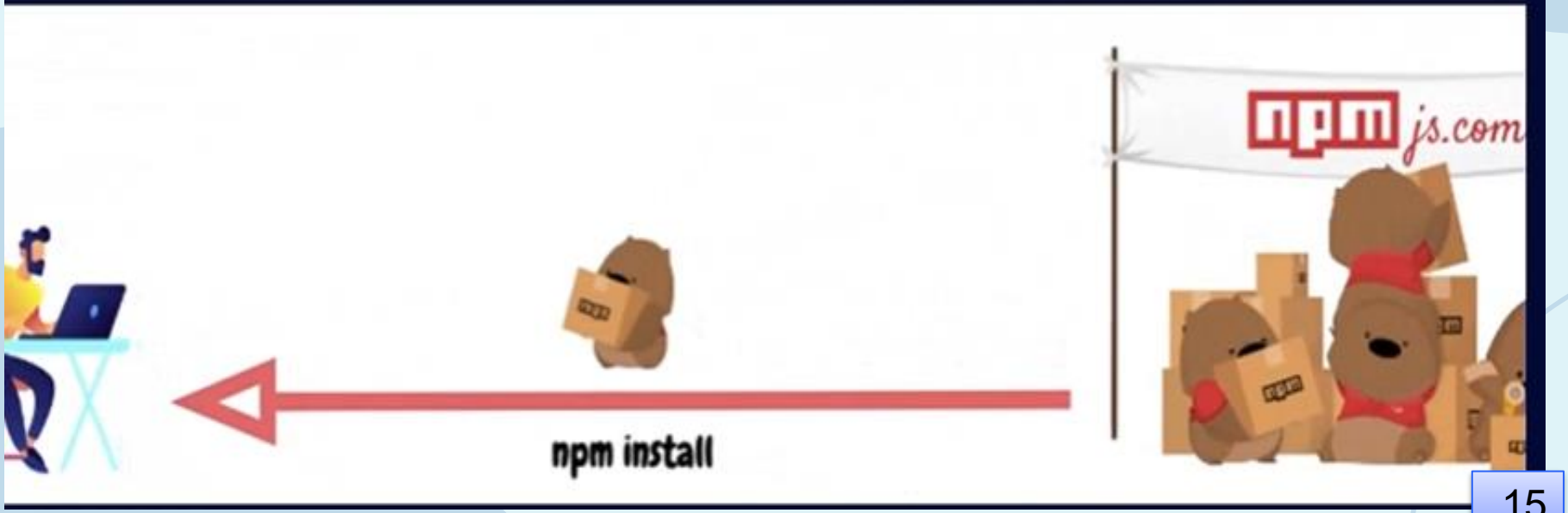


Quelques outils utilisés par React :

- **npm (node package manager) :** le gestionnaire de paquets par défaut pour une application JavaScript

- **webpack :** bundler JavaScript (pour construire le graphe de dépendances et regrouper des ressources de même nature (.js ou .css...) dans un ou plusieurs bundles)
- **Documentation officielle :** https://fr.reactjs.org/

NPM: NODE PACKAGE MANAGER



Environnement du travail et Installation

- Installation des outils :
  - Node JS :
    - Site : https://nodejs.org/en/download/
    - Node JS installe l'outil npm (Node Package Manager) qui permet de télécharger et installer des bibliothèques Java Script : gestionnaire de modules de Node.
    - **Télécharge la version LTS (Long Term Support).**
    - Pour vérifier les versions installées :

```
> node --version ou node -v (version de node)
> npm --version ou npm -v (version de npm)
> npm view react version (version de React)
```

Environnement du travail et Installation

- Installation des outils :
  - **Editeur :**
    - Visual Studio Code (https://code.visualstudio.com/) : gratuit et s'étend selon le langage de programmation
    - Autres éditeurs : IntelliJ, WebStorm, PHP Storm, ...
    - **Extensions :**
      - Installation des plugins (extensions) y associés est souhaitable afin de faciliter le codage. (Exemple : Simple React Snippets sous VSC : imr, imrc, cc , ccc, ffc, ...)
  - **React DevTools :** Extension sur chrome et firefox qui permet d'examiner :
    - une arborescence de composants React dans les outils de développement du navigateur
    - les props et l'état local des composants React

Lancement d'un projet React

- Création d'un projet React avec CRA et Vite :
  - ❖ **Méthode 1 : Create React App (CRA)**
    - **Create React App (CRA) :** `create-react-app`
      - Outil pour faciliter le développement d'applications web fondées sur React
      - Eviter les problèmes d'installation, de configuration et d'intégration
      - Génération automatique d'un squelette applicatif

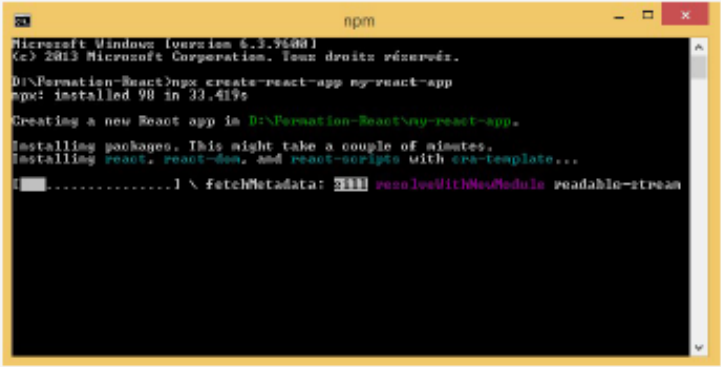
Au préalable, pour installer l'outil create-react-app :

```
> npm install -g create-react-app
```

Lancement d'un projet React

- Etapes à suivre :
  - Créer un nouveau dossier « D:/Formation-React »

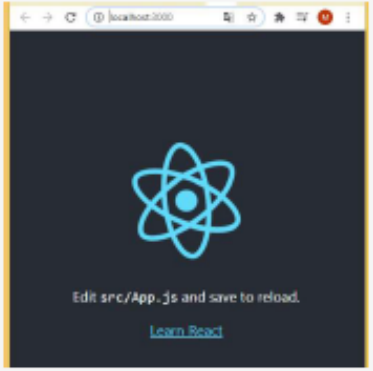
```
> npx create-react-app my-react-app
```




```
> cd my-react-app

> npm start
```

L'application se lance par défaut en local sur :
http://localhost:3000/



Lancement d'un projet React

❖ **Méthode 2 : Vite (plus rapide)**

- npm create vite@latest mon-app-vite
- cd mon-app-vite
- npm run dev

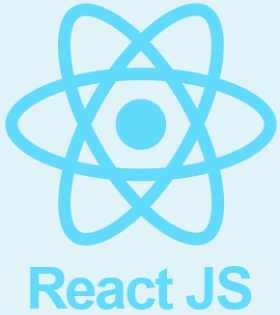
Vite + React

```
C:\Users\asus\Desktop\Projet-react>npm create vite@latest mon-app-vite
```

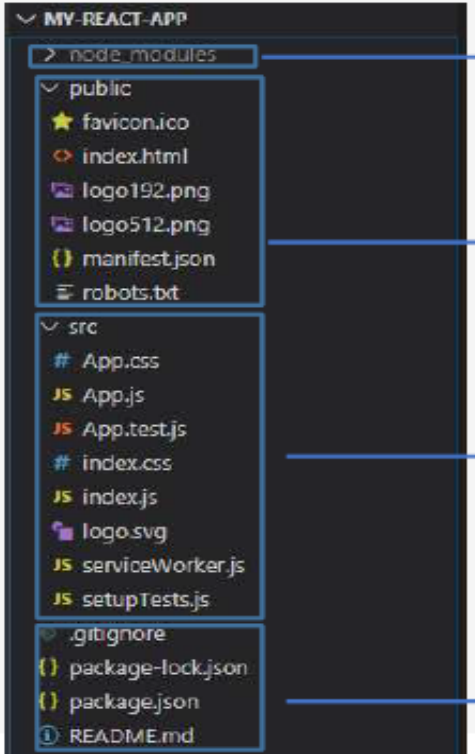
Ou

npm create vite@latest mon-react-pro -- --template react

Structure d'une application ReactJS



Structure d'une application ReactJS



Dépendances externes (librairies) de l'application

Ressources publiques de l'application

Code source de l'application (composants du react : jsx, css, ...)

Fichiers de configuration du projet

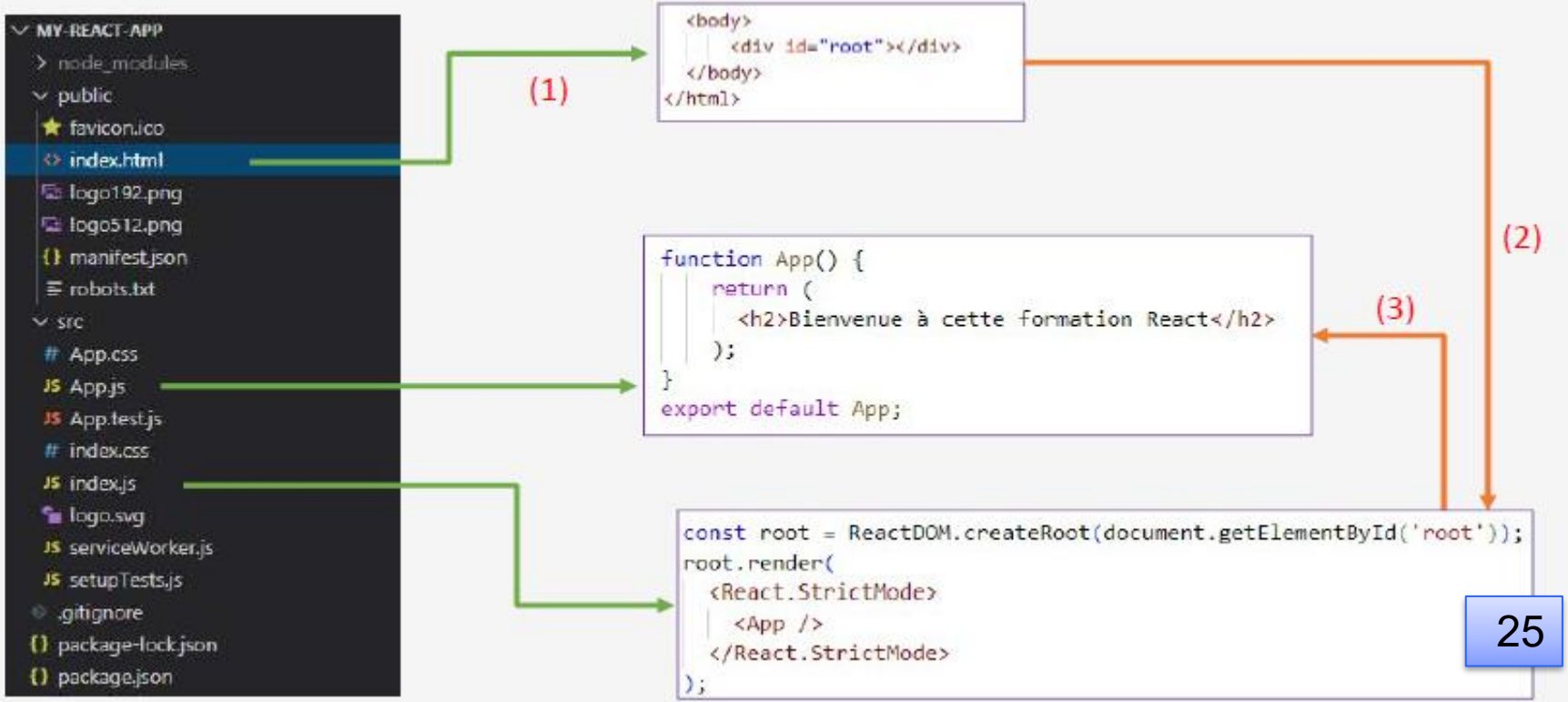
Structure d'une application ReactJS

- Principaux répertoires et fichiers d'une application React :
  - **nodes_modules :** Toutes les bibliothèques Java Script externes utilisées par l'application
  - **index.html :** la racine de l'application (en spécifiant l'identifiant de l'élément qui sera affiché par le navigateur)
  - Le répertoire **public/ :**
    - fichiers HTML, JSON et images de base (les racines de l'application).
  - Le répertoire **src/ :**
    - fichiers sources (js, css, etc.) que nous créerons devront être dans ce répertoire
    - De préférence, tous les composants (components) qui vont être définis par la suite devront être dans un dossier à part (à titre d'exemple : ***components***).

Structure d'une application ReactJS

- Principaux répertoires et fichiers d'une application React :
  - **index.js :** le point d'entrée d'application incluant le code qui sera injecté au niveau du fichier de *index.html*
  - **App.js :** le composant principal (root component)
  - Au niveau de CRA (Create React Application)
    - Seuls les deux fichiers src/index.js et public/index.html sont exigés
    - Le reste des fichiers est optionnel
  - **package.json :** fichier de configuration
    - contient les métadonnées du projet (titre, version, ...) ainsi que toutes les dépendances nécessaires afin de pouvoir créer et exécuter une application React.

Structure d'une application ReactJS



Que fait index.js en React ?

```
root.render(
    <React.StrictMode>
      <App />
    <React.StrictMode>;
);
```

root.render() est une **méthode** qui sert à : Afficher un composant React dans la page web.

React.StrictMode

Active des vérifications supplensentaires

- Détecte les érreurs potentiuales
- Aide a écrire un code plus sécurisé
- signaler les mauvaises pratiques
- aider à écrire du code propre

Composant Principal <App />

Le composant principal de votre application

root.render()

Affiche l'application dans le DOM

- Insére l'application dans la page web

Affiche l'application en mode strict avec le composant <App />

Structure d'une application ReactJS

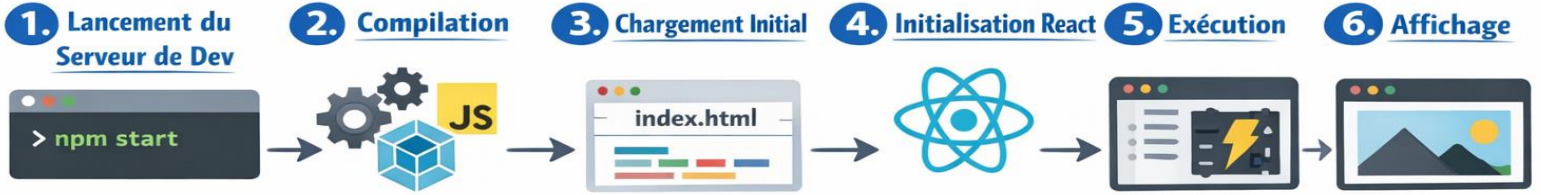
- **ReactDOM.createRoot :**
  - Avec les nouvelles fonctionnalités de React 18, il s'agit d'une nouvelle méthode pour créer une racine à rendre ou à démonter à la place de *ReactDOM.render*.

- **React.StrictMode :**
  - outil pour détecter les problèmes potentiels d'une application
  - activer les vérifications et les avertissements supplémentaires, uniquement durant le développement pour les composants descendants.

```
<React.StrictMode>
    <ComponentOne />
    <ComponentTwo />
</React.StrictMode>
```

Flux de Chargement EXACT de React

Que se passe-t-il quand tu lances : **npm start** ?



- Démarrage de Webpack Dev Server
- Compilation du code source

- Transpilation avec Babel
- **Bundling** avec Webpack
- Code préparè pour le navigateur

- **index.html** chargé
- JS & CSS injectés
- Fichiers statiques chargés

- ReactDOM.render(),
- Montage du composant racine
- Création du Virtual DOM

- Exécution du code JS
- Hooks & Effets lancés
- Mise à jour du DOM

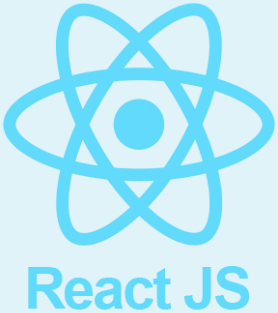
- **Rendu** à l'écran
- Interaction utilisateur
- Mises à jour dynamiques

- src/index.js s'exécute

React affiche <App />

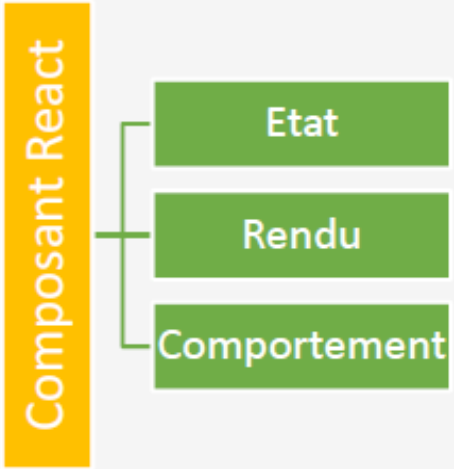
Ton appli React est préte et tourne ! ✅

Développer avec React JS : Principes de base



Les fondamentaux du React

- **React :**
  - permet de créer des applications web de type SPA (Single Page Application)
  - basé sur une approche de programmation orientée composants (Web Components)
- **Composant React est caractérisé par :**
  - L'**état** du composant (State) qui représente le Modèle
  - Le **rendu** : Structure de la vue du composant (View)
  - Le **comportement** (Behaviour ou Controller)



Site classique (Multi Page Application)

Une **MPA** charge une nouvelle page HTML à chaque action de l'utilisateur qui change de page.

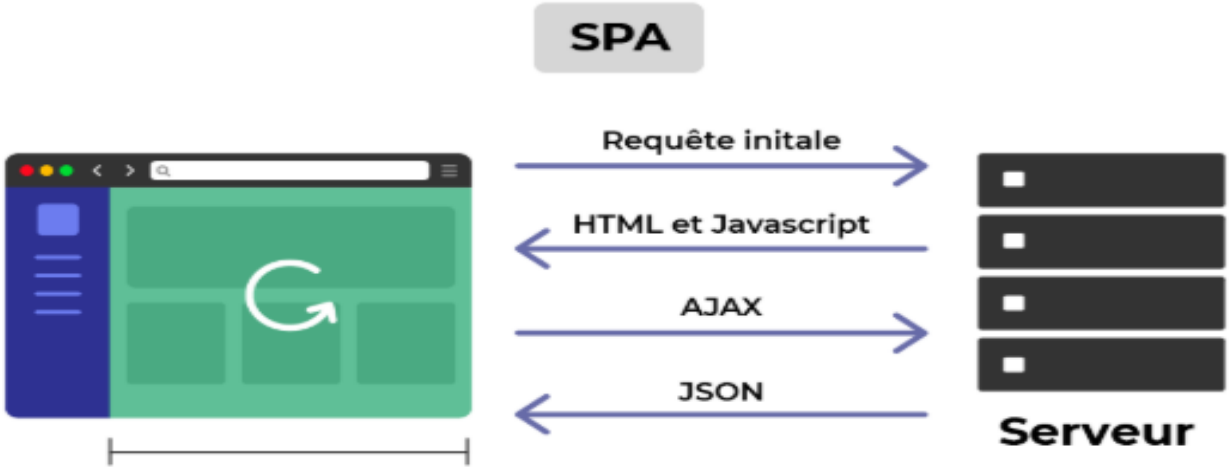
Caractéristiques :

- Chaque page a son **propre fichier HTML**.
- Le serveur renvoie une nouvelle page complète à chaque navigation.
- La navigation provoque un **rechargement complet** du navigateur.
- Plus simple côté SEO car chaque page est servie par le serveur.

Que signifie SPA (Single Page Application)

Une **SPA** est une application web qui :

- **Ne recharge pas** toute la page à chaque action
- **Charge une seule fois le HTML**
- Met à jour uniquement le contenu nécessaire



Comment fonctionne une SPA en React ?

Une **SPA** est une application qui charge **une seule page HTML** et met à jour le contenu **dynamique** sans recharger toute la page.

En React :

- React gère le **DOM virtuel (Virtual DOM)** pour mettre à jour seulement les parties nécessaires de la page.
- On utilise des **composants** pour organiser l'interface.
- La navigation se fait **côté client** via **des bibliothèques** comme **React Router**.
- Le serveur fournit **uniquement les données**, souvent en JSON via des API REST ou GraphQL.

Comment fonctionne une SPA en React ?

Avec **React**, une SPA fonctionne ainsi :

- Le navigateur charge une seule fois index.html
- React prend le contrôle du
- Toute l'interface est générée dynamiquement via JavaScript
- La navigation change les composants affichés, sans rechargement complet

Comparaison : Site classique vs SPA

Critère	SPA (React)	MPA
Nombre de pages HTML	Une seule	Plusieurs
Rechargement	Non, contenu dynamique	Oui, chaque navigation recharge la page
Navigation	Côté client (React Router)	Côté serveur
Performance UX	Très rapide	Moins rapide
Complexité	Plus complexe à mettre en place	Plus simple à coder
SEO	Plus compliqué (nécessite SSR ou prerendering)	Simple, le serveur rend chaque page

Pourquoi on utilise SPA ?

❑ **Performance**

Moins de rechargement complet.

❑ **Expérience utilisateur (UX)**

Navigation fluide.

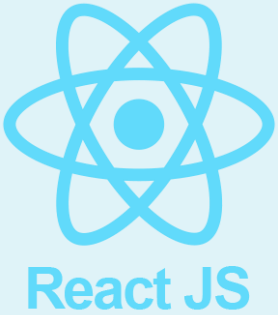
❑ **Moins de charge serveur**

Le serveur renvoie surtout des données (JSON).

Idéal pour :

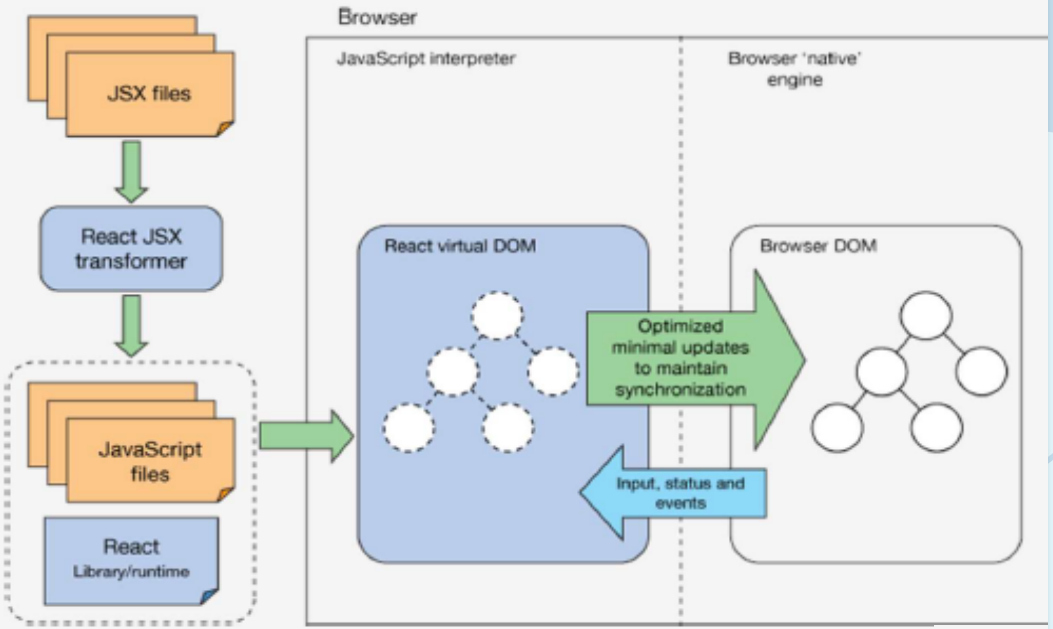
- Dashboard
- Application bancaire
- E-commerce
- Réseaux sociaux

Architecture React



Architecture React

- Le moteur de rendu JSX, permet de coupler dans un même fichier JSX, les trois aspects d'un composant React
- Le moteur de React génère un DOM virtuel qui sera transformé et synchronisé avec le DOM réel du Navigateur
- Le Virtuel DOM optimise les changements à apporter au niveau du Browser DOM. Ce qui lui permet d'accélérer l'aspect réactif du rendu.



Real DOM (DOM réel)

- Le **DOM réel** est **la représentation en mémoire de la page HTML que le navigateur utilise pour afficher le site**.
- Chaque élément HTML (div, p, img…) est un **nœud dans ce DOM**
- Le navigateur lit ce DOM pour **dessiner la page à l'écran**

Exemple :

```
<div id="app">
  <h1>Hello</h1>
  <p>Bienvenue</p>
</div>
```

- Le navigateur crée **un arbre d'objets** en mémoire représentant ces éléments.

Real DOM (DOM réel)

- Le **DOM (Document Object Model)** est la **représentation de votre page HTML** dans le navigateur.
- Quand vous modifiez quelque chose dans le DOM (ex : changer un texte ou ajouter un élément), le **navigateur doit recalculer toute la page ou les parties concernées**.
- Cela peut **ralentir** la page si beaucoup d'éléments changent.

```
<div id="root">
  <p>Bonjour !</p>
</div>
```

Si vous changez "Bonjour !" en "Salut !", le navigateur met à jour directement ce paragraphe dans le DOM réel.

⚠️ Problème : si la page est grande, chaque changement direct peut être **lent et coûteux en ressources**.

⚠ Problème du DOM réel

- Quand tu modifies un élément, le navigateur **recrée ou met à jour la partie de la page**
- Cela peut être **lent si la page est grande**

Exemple :
si tu changes un petit texte dans un tableau énorme, le DOM réel peut recalculer tout le tableau → inefficace

virtual DOM (DOM virtuel)

React crée une **copie légère du DOM réel** appelée **Virtual DOM**.

- Quand l'état d'un composant change (state), React **modifie d'abord le Virtual DOM** au lieu du DOM réel.
- React **compare le Virtual DOM avec le DOM précédent** pour voir exactement ce qui a changé.
- Puis React **met à jour uniquement les parties modifiées du DOM réel** (diffing).

Avantages :

- Optimisation des performances
- Moins de calculs pour le navigateur
- Mise à jour plus rapide et fluide de l'interface

Exemple virtual DOM (DOM virtuel)

```
function App() {
  const [count, setCount] = React.useState(0);

  return (
    <div>
      <h1>Compteur : {count}</h1>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

Chaque clic crée un **nouveau DOM virtuel**

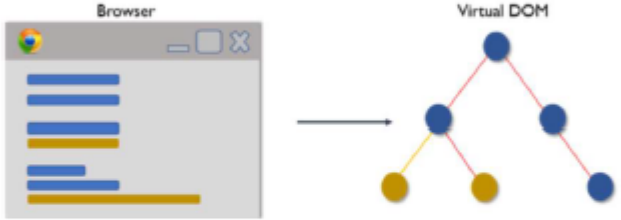
•React compare avec l'ancien

•Il modifie **seulement le** <h1>, pas tout le → plus rapide

Virtual DOM

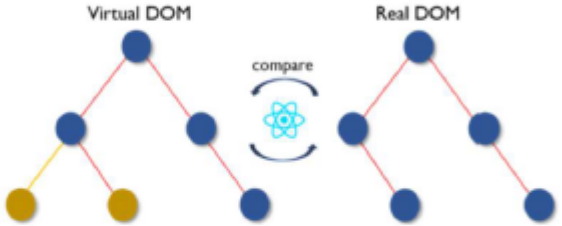
Etape 1 :

- Chaque fois que des données changent, l'interface utilisateur entière est restituée en représentation Virtual DOM



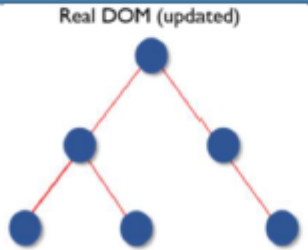
Etape 2 :

- La différence entre la représentation DOM précédente et la nouvelle est calculée



Etape 3 :

- Une fois les calculs effectués, le DOM réel va mettre à jour uniquement les objets qui ont réellement changé.



Le Virtual DOM en React

Qu'est-ce que c'est ?

Virtual DOM

Copie légère du DOM réel

DOM Réel

Structure du site

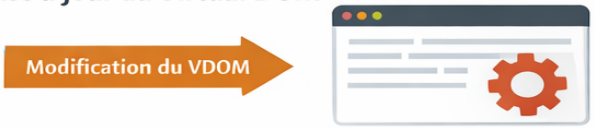
Permet de simuler les changements.

Fonctionnement

1. Comparaison des arbres

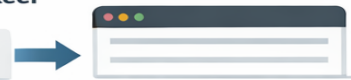
— Différences détectées —

2. Mise à jour du Virtual DOM



3. Mise à jour du DOM Réel

Mise à jour optimisée



Les Avantages

Performances améliorées

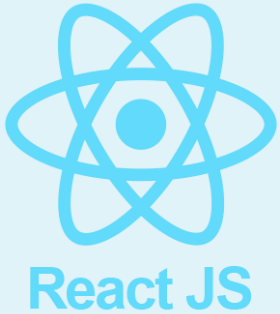
Mises à jour rapides

Moins de ressources utilisées

Real DOM Vs Virtual DOM

Real DOM	Virtual DOM
Il se met à jour lentement.	Il se met à jour plus rapidement.
Peut directement mettre à jour le code HTML	Impossible de mettre à jour directement le code HTML.
Crée un nouveau DOM si l'élément est mis à jour.	JSX s'occupe de la mise à jour de l'élément
La manipulation DOM est très coûteuse.	La manipulation DOM est très facile.
Gaspillage de la mémoire.	Bonne gestion de la mémoire.

data binding



Principe de data binding

Contrairement à certains frameworks (comme Angular), React utilise un :
One-Way Data Binding (liaison de données unidirectionnelle)

Cela signifie :
Les données circulent **dans un seul sens** :

State / Props → UI (JSX)

Principe de data binding

Dans React :

- Les **données (state)** sont stockées dans le composant
- Ces données sont **affichées dans le JSX**
- Si les données changent → **l'interface se met à jour automatiquement**

⇨ React fait ça grâce au **Virtual DOM**

Props → Data Binding entre composants

React permet aussi de passer des données :

👉 **Parent → Enfant (props)**

```
function Parent() {
  return <Enfant nom="Arij" />;
}

function Enfant(props) {
  return <h1>{props.nom}</h1>;
}
```

👉 Ici :

- Parent → props → Enfant → UI
- ✅ Toujours unidirectionnel

Et le Two-Way Binding ?

- React **n'a pas de two-way binding automatique**, mais on peut le simuler :
  Exemple avec un input :

```
function Formulaire() {
  const [name, setName] = useState("");

  return (
    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
  );
}
```

Ce qui se passe :

1. `value={name}` → state → input (affichage)
2. `onChange` → input → state (mise à jour)

👉 Donc : `State ↔ Input`

👉 **contrôlé manuellement (controlled component)**

✅ Avantages de One-Way

🔒 Prévisible (moins de bugs)
🧠 Facile à comprendre
⚡ Performant (Virtual DOM)
🔧 Debugging simple

Merci pour

votre Attetion